`BivariateAggregate` Class Cheat Sheet

m `BivariateAggregate(name, units=None, copula=None, note='', hints='', tags={}, doc='', mode='copula', nc_agg, nc_kwargs, nc_views, clash, dbv_xs, dbv_ys, dbv_S, exp_en, exp_el, exp_premium, exp_lr, freq_name='poisson', freq_a, freq_b, freq_zm, freq_p0)`

A `BivariateAggregate` is the joint law (A_0, A_1) of two coupled aggregates. A **shared** outer frequency drives both; within each event the two per-claim severities are coupled, and the joint is accumulated by a 2D FFT. Either marginal reproduces the standalone `Aggregate`. Usually built via `build('bivariate ...')` (alias `bv`); see the DecL card, page 3. The following tables show all m `methods`, and fields or properties (used interchangeably). Comments elucidate the meaning of more obscure entries.

1. Specification & creation

`name`, `units` (the two component `Aggregates`), `unit_names`, `copula` (the fitted `Copula`), `frequency`, `freq_name` (the *shared* outer count), `en` (expected event count), `clash` (the solved $(n_a, n_b, n_c, n_0, p_a, p_b)$ when declared with `clash`), `program`, `pprogram`, `mode`[1]

Four declaration modes: `copula` (two `agg` bodies + a `copula` clause), `discrete` (`dbvsev`, the joint per-claim matrix given directly), view-pair (`netceded` / `grossceded` / `grossnet` over one reinsured `agg`), `clash` (counts n_a, n_b, n_c solved into a shared count and two Bernoulli triggers).

2. Update

m `update(log2, bs, padding, store_dir, row_chunk, col_chunk, keep_transform)`, m `update_work`, `bs` (per-axis bucket sizes, a *list*), `axis_xs` (per-axis output grids), `padding`

Scalar `log2` is the total *2-D cell budget, split between the axes by measured support; a 2-tuple pins the per-axis `log2`. `store_dir` switches to the* massive *disk-backed path (extra `aggregate[massive]`), where `padding` defaults to 0.*

3. Moments

m `moments` (mixed raw moments $\mathsf{E}[A_0^i A_1^j]$), `corr` (Pearson correlation of the two aggregates), `dependency_df` (`cov` / `corr` / `tau` at both levels: per-claim `Sev` and `Agg`), `stats_df` (per-component marginal moments, theory vs. empirical)

4. Statistical functions

Via the `bivariate` *view*[2]: m `pushforward` (law of $z = f(A_0, A_1)$ on a fresh 1-D grid — the workhorse: *any* scalar function of the pair), m `transformed_moments` (exact raw and central moments of $f(A_0, A_1)$, no regridding), m `moments`, m `corr`, m `marginals`

Univariate risk measures come from the pushforward or from either marginal `Aggregate`; a 2-D law has no single quantile.

5. Validation

`summary_df` (two-unit `Portfolio`-shape summary: theory vs. realised), `validation_explanation`, `deficit` (mass lost off the grid corner)[3], `bs_window_df`, `bs_description`, `bs_explanation` (per axis), `tail_df`, `tail_description`, `tail_explanation`

6. Output dataframes

`density` (the joint pmf, shape (n_0, n_1)), `density_df` (the same as a `DataFrame`), `marginals` (the two 1-D densities), `bivariate` (the `BivariateDistribution` view)[2], `info`, `summary_df`, `stats_df`, `dependency_df`, `axis_xs`, `figure`

7. Reinsurance

The whole point of the view-pair modes. `netceded` `<AGG>` (and `grossceded`, `grossnet`) build the joint occurrence law of two of {gross, ceded, net} for one reinsured aggregate — exact dependence, comonotone within the layer. The object-API route is m `Aggregate.occ_bivariate(views)`.

Per-component reinsurance is declared on each inner `agg` line as usual.

8. Visualization

m `plot` (two-panel contour: per-claim severity, then aggregate), m `bivariate.contour` (one filled contour panel), `figure`

9. Risk and pricing

None directly. Push the joint through the payoff you want to price — `bivariate.pushforward(f)` returns an ordinary 1-D distribution — then price that, or wrap it in a `PnL` (a `BivariateDistribution` is an accepted P&L `source`).

10. Approximations

None. The 2-D FFT is exact on the grid; the only approximation levers are the axis budgets (`log2`, `bs`) and `padding`, in Update.

11. Meta

m `help`, m `format_program`, `pprogram_html`, `note`, `tags`, `doc`, `hints` (the DecL trailer)

Notes:

[1]: `mode` is `'copula'` or `'discrete'`; the view-pair and `clash` statements set the remaining constructor arguments (`nc_agg` / `nc_views`, `clash`).

[2]: `bivariate` is a `BivariateDistribution`: `density`, `axis0` / `axis1` (positionally `ceded` / `net`), `axis_names`, `bs_ceded` / `bs_net`, `meta`, and m `marginals`, m `moments`, m `corr`, m `contour`, m `pushforward`, m `transformed_moments`. Address axes *by role* (`axis0` / `axis1` / `axis_names`), never by the positional names.

[3]: With `padding=0` the FFT wraps rather than losing mass, so `deficit` can read clean while the answer is wrong: compare means, not `deficit`.

`aggregate v.1.0.0a171`
ease | confidence | precision

2026-07-29 11:36